

Aims

- Create a PHP file that processes user input from a form and returns a response to the user.
- Sanitize form data at the server.
- Check data for text, drop-down boxes, radio buttons and checkboxes.
- Use regular expressions in PHP.
- Force the Web page to redirect if a user tries to enter the PHP processing page directly via the URL.

Note: All delivered web pages should conform to HTML5 and must be validated.

Note: To validate that the code generated by your PHP script is valid HTML5 you must view the code received by your browser. You cannot successfully validate HTML directly from PHP source file.

Task : Checking user input data at the server

In this task we will create a script that instructs the PHP engine on the **server** to process data sent from a client webpage. The data will be *name-value* pairs sent from a form that a user has entered and submitted. A webpage that we construct from a mixture of HTML and PHP will then send back a confirmation to the user.

The webpage will look something like this:

Rohirrim Ranch Tours

Tour Booking Confirmation page

Booking Details

Welcome John Mike

Your booking is on the following package(s):

1 day tour and 10 days tour

Species: Elf

Age: 22

Meal preferences: Ent Draft

Number of travellers: 2

Step 1: Load an example HTML form

In this step we use the same HTML form that we used in Week 7 lab. This time however, we will be validating the input at the **server** using **PHP** rather than at the **client** using HTML5/JavaScript.

1. Create a folder **lab08** at htdocs (C:\xampp\htdocs), copy the file from **lab08.zip** to the folder. This zip file contains a file called **register.html**. Rename to **register.php**.
2. We need to direct where the information collected from the form will be submitted. We will submit with method **post** to **confirm.php**.

Add the following **attributes** to the <form> element of **register.php**.

method="post" action="confirm.php"

3. So we can check the PHP data validation is working, we will turn off HTML5 data validation by **novalidate = "novalidate"** adding the attribute to the form.
4. Change the checkbox's name and value to the following:

```
<legend>Your trip:</legend>
<fieldset>
  <legend>Booking:</legend>
  <label for="1day">1 Day Tour - $200 </label>
  <input type="checkbox" name="book[]" id="1day" value="1 day tour" /><br />
  <label for="4day">4 Day Tour - $1500</label>
  <input type="checkbox" name="book[]" id="4day" value="4 days tour" /><br />
  <label for="10day">10 Day Tour - $3000</label>
  <input type="checkbox" name="book[]" id="10day" value="10 days tour" /><br />
</fieldset>
```

5. View **register.php** in your browser.

Step 2: Create a PHP file to process the form

A PHP file is an HTML file with some embedded script. The PHP processor on the server will process it and generate the HTML, in the file, where embedded scripts are located.

PHP code in an HTML file can generate *part* of the HTML or *all* of the HTML. (or even *none* of the HTML – but then there is not much point giving it a PHP extension!)

We will start by creating a standard HTML template, then add some PHP script in the next step.

1. Using Notepad++ or similar, create a standard HTML template with a doctype instruction, header, and body elements. (Check out lab02 again if you have forgotten what should go in the HTML header).
2. Give the page the title "Booking Confirmation".
3. In the <body> add a heading element that says "Rohirrim Ranch Tours" and level 2 heading element that says "Tour Booking Confirmation page".
4. Save it as **confirm.php** and save it in the same folder you saved **register.php**(step 1).

Step 3: Add some PHP to the HTML process file to extract values from the form

When a **submit** button is actioned on a form, the *name-value* pairs are sent from the client to the server through a **GET** or a **POST HTTP** request message. This request message is then processed according to the PHP code in the file specified in the **action** attribute. In Lab 7 task 2, we sent a single *name-value* pair representing an integer number to a PHP process file via a **GET method**.

In this step, we will send all the *name-value* pairs from the form on **register.php** to be processed by the file **confirm.php** that we created in Step 2. As we are using a **POST** message to send this data, we need to use the **\$_POST** autoglobal to extract the *values* associated with each *name*.

We can use the in-built PHP function *isset()* to check if a named value in the **POST** message has been defined. It will not be defined if the process was not triggered by a form *submit*, for example if someone attempts to access the PHP directly by typing the URL of the PHP file.

1. In the body element, after the heading, create a PHP block

```
<?php
?>
```

2. Inside the PHP block, create an *if-else* statement as shown below. If the condition in the *if-else* statement evaluates to true, we will store the value from the form into a variable. If the value is not set we will print an error message on the Web page.

```
<body>
  <h1>Rohirrim Ranch Tours</h1>
  <h2>Tour Booking Confirmation page</h2>
  <?php
    //assign the variables to empty here

    //checks if process was triggered by a form,
    //if not display an error message on another file

    if (isset ($_POST['firstname'])) {
      $firstname = $_POST['firstname'];
    }
    else {
      //using require make link to error.php
    }

    //assign the rest of the form validation to PHP variables here
```

3. Upload **confirm.php** to the htdocs folder at C drive. Check that it is saved in the same folder you save **register.php**.
4. Open **register.php** in your browser and make sure it echoes back the name you entered in the first name field.
5. Immediately below the *if-else* statement, test if lastname is set, then create a variable **\$lastname** and assign the appropriate **\$_POST** autoglobal variable to it as follows:

```
if (isset ($_POST["lasstname"])) { $lastname = $_POST["lastname"]; }
```

6. In the same way, create and initialize variables for **\$book**, **\$species**, **\$age**, **\$food** and **\$partysize**.
7. Add a statement to **echo** the variables to HTML page so we can check they have been correctly assigned.
8. Save and upload **confirm.php** again to htdocs folder.

Hint: checkbox will need to use implode method.

Step 4: Create an error message file.

We will start by creating a standard HTML template, then save it as error.php.

1. Using Notepad++ or similar, create a standard HTML template with a doctype instruction, header, and body elements.
2. Give the output as below:

Error Messages

You have missing/empty fields

Please click this [link](#) to return to register page

3. Make the link back to register page (register.php).
4. Validate the HTML generated by your PHP script.

Reminder: To validate that the HTML generated by PHP is valid HTML5, we cannot directly upload the PHP to the validator. The PHP script must be processed by the PHP pre-processor on the server, create the HTML that is sent to the client. However we can view the source of the HTML page displayed in the browser, then cut and paste it to the W3C HTML validator <http://validator.w3.org/nu/>